ROB-10

TEACHER GUIDE

Robotics · Motors & Motion Control

Balancing Robot

Segways stay upright by falling forward and catching themselves 100 times a second.

1 At a glance

LENGTH	TYPE	GROUPING	CONCEPTS
120 minutes	Hardware + software	Pairs (larger groups of 3 to 4 if kits are short)	inverted pendulum · sensor fusion · PID

2 Overview

The capstone of the motors module. Students feel an inverted pendulum by balancing a broom, learn why an accelerometer is noisy and a gyro drifts, and fuse the two with a one-line complementary filter. They tune a PID controller in the PID Lab simulator where nothing can break, then rebuild the ROB-05 robot tall, close the loop for real, and tune it one gain at a time with a written log.

> The one idea that must land
>
> A controller only ever sees the error. P pushes against the lean, D brakes, I clears up what is left, and you change one gain at a time and write it down. Fast, steady measurement (a fixed 10 ms loop) matters as much as the gains.

3 Learning outcomes

By the end, students can:

1. Read **tilt** from an MPU6050 and explain why one sensor is not enough.
2. Combine accelerometer and gyro with a **complementary filter**.
3. Balance an **inverted pendulum in simulation** and tune it.
4. Build and tune a real **two-wheel balancing robot**, with a log that explains the tuning.

4 Three depths

Pick one for the class. Everyone does the same lesson; only the build target changes.

Starter

Balance a broom on your hand and spot the pattern

Balance a broom on your hand and spot the pattern, then show accelerometer noise and gyro drift on the Serial Plotter, then tune the pendulum in the PID Lab. The real robot is demonstrated by the teacher or a faster pair.

Core

Simulated pendulum, then the real robot with a given PID

Full lesson: read and fuse the angle, tune in the PID Lab, then run the real robot starting from the given gains and improve it with a tuning log.

Advanced

Tune the robot and add remote steering

Tune the real robot to recover from a standard nudge, then add remote steering by shifting the setpoint a degree or two, or add encoders so it holds its position.

5 Before the lesson

Kit, per group of 2

- MPU6050 IMU module
- The pair's ROB-05 robot: 2WD chassis, L298N, Arduino Uno + USB cable
- **Two** 18650 cells in a 2-cell holder (7.4 V). One cell (3.7 V) is not enough: the L298N loses about 2 V, leaving the motors almost nothing.
- Standoffs or a stiff card tower to mount the battery high
- Male-female jumper wires
- A soft surface to test over (a jumper or foam mat)

Simulators

- **PID Lab** (Simulators → PID Lab, pendulum mode): tune P, I and D on a simulated pendulum with an instant response graph.
- **Code Lab**: try the complementary filter maths on made-up readings in Python first.

Prep checklist

- [ ] Charge the 18650 cells fully and check their holders. Inspect every cell: dented or torn wrappers are not used.
- [ ] Load the complete angle sketch from the end of this guide onto one robot and check the angle reads about 0 when upright. Slide 12's code is the core of it; students need the full version.
- [ ] Open the PID Lab on every laptop in pendulum mode before the lesson.
- [ ] Prepare one robot as a demonstration, already tuned, in case the class runs short of time.
- [ ] Print or draw the tuning log table (slide 15) for each pair.

6 Timing plan

Min	Phase	What happens
5	Let's think	Balance a broom; the four guesses (slide 2).
3	Aims	The four aims and keywords (slide 3).
27	Teach	Two sensors, the filter, error and PID, bad gains, real examples (slides 4 to 8).
5	Kit and the tall build	Why the battery goes high; safety (slides 10, 11).
15	Read the angle	Accelerometer, gyro and fused angle compared (slide 12).
20	Tune in the PID Lab	P until it wobbles, halve it, add D, nudge test (slide 13).
30	Close the loop for real	True zero, half the sim gains, one change at a time, the log (slides 14, 15).
10	Prove it is controlling	Nudge, book on top, Kd to zero (slide 16).
5	Check and exit ticket	Self-rating; the shaking robot question (slides 18, 19).

Total: 120 minutes

7 Step by step

Let's think

5 min · slides 1-2

Say: Stand a ruler or broom on your palm and keep it up. Which way does your hand move?

Say: Now try it with your eyes shut. Without measurement there is no control.

→ Students try it, then answer the four guesses: long vs short broom, reacting too strongly, reacting too late, why not a pencil.

Look for: students noticing they react to the lean, not the position, and never stop moving. That is the whole controller.

Aims

3 min · slides 3

Say: Today a robot stands on two wheels by falling forward and catching itself a hundred times a second.

Teach

27 min · slides 4-8

Say: The accelerometer knows which way is down but jumps with every vibration. The gyro is smooth but drifts. Trust the gyro for the short term and gravity for the long term.

Say: `angle = 0.98 × (angle + gyro × dt) + 0.02 × accelAngle`. Mostly carry on from where I was, but nudge 2% towards what gravity says, every time.

Say: error = 0° minus the measured angle. The controller never thinks about upright; it only pushes a number towards zero.

Say: P pushes in proportion to the lean. D brakes by reacting to how fast the lean is changing. I clears up a small lean that never goes away.

- → Slide 5: read the filter line aloud together.
- → Slide 7: draw the two bad shapes: too much P shakes, too little P leans and falls.
- → Slide 8: everyone stands still and feels their ankles correcting.

Look for: the belief that bigger gains are always better. Too much P is exactly the shaking robot in the exit ticket.

Kit and the tall build

5 min · slides 10-11

Say: A long broom is easier than a pencil because it falls more slowly. So we build the robot tall and put the heavy battery at the top.

- → Rebuild the ROB-05 robot upright with the battery high. Two 18650 cells in series, 7.4 V.
- → Wire the MPU6050 to 5V, GND, A4 (SDA) and A5 (SCL), mounted firmly; a loose sensor reads vibration as tilt.

Read the angle

15 min · slides 12

Say: Three numbers, three behaviours. Watch the accelerometer jump when you tap the desk, and the gyro wander when you leave it still.

- → Upload the complete angle sketch (end of this guide). Open the Serial Plotter.
- → Tilt, tap and leave still; compare raw accelerometer, gyro-only and fused angles.
- → Leave a gyro-only reading running to see the drift.

Look for: an angle that does not change when the robot tips forward. The sensor is mounted on its side: use `ax` and `gy` instead of `ay` and `gx`.

Tune in the PID Lab

20 min · slides 13

Say: In the simulator a bad gain costs nothing. On the desk it costs a snapped bracket.

- → Pendulum mode. I and D at zero; raise P until it wobbles forever.
- → Halve P, then raise D until the wobble dies.
- → Nudge it: it should recover in under a second. Write the three numbers on the board.

Look for: pairs comparing numbers and worrying they differ. That is correct: gains belong to the machine.

Close the loop for real

30 min · slides 14-15

Say: Hold it upright and read the angle. That number is your real zero, not 0.

Say: Change one gain, try again, write down what happened. Two changes at once and you will never know which worked.

→ Set the setpoint to the measured upright angle.

→ Start from the slide's gains (Kp 22, Ki 0.4, Kd 1.4) or half the PID Lab numbers.

→ Hold it, power on, let go slowly over the soft surface. Log every try.

Look for: the 35° cutout working. A robot past 35° cannot recover; the motors must stop rather than thrash.

Prove it is controlling

10 min · slides 16

Say: Working once is not the same as controlling. Make it prove it.

→ Nudge it; add a book on top; set Kd to zero and watch the wobble return.

Check and exit ticket

5 min · slides 18-19

Say: Your tuning log is your proof. It should explain why your robot stands.

→ Self-rate.

→ Exit ticket: the robot shakes rapidly and falls. Which gain first, up or down, and why?

8 Common misconceptions

Students may think	How to help
The accelerometer alone gives the angle.	It does when still, but motor vibration makes it jump. It needs the gyro for the short term.
The gyro alone gives the angle.	It is smooth, but adding up its readings collects error: it drifts a degree every few seconds.
Bigger gains mean better balance.	Too much P overshoots and shakes the robot over. Tuning is finding the right size, not the biggest.
A short robot is easier to balance.	Shorter falls faster. Taller with the weight high gives the controller more time.
Change all three gains to fix it quickly.	Change one at a time and log it, or you will never know which change worked.

9 Troubleshooting

Problem	Fix
Angle does not change when it tips	The sensor is mounted on its side. Swap to the other axis (ax and gy), or rotate the sensor.
Angle drifts away while still	The accelerometer part is not mixing in. Check the 0.02 term and that ay/az are read correctly.

Angle jumps around	Loose sensor mount, or loop time not steady. Fix the sensor firmly and keep a fixed 10 ms loop.
Shakes fast and falls	Kp too high, or Kd too low. Lower Kp first.
Leans slowly and falls	Kp too low. Raise it until it just starts to oscillate, then add D.
Motors weak or barely turn	One 18650 is not enough. Use two in series (7.4 V), fully charged.
Drives off in one direction while upright	The setpoint is not its true upright angle. Measure it and use that, or add a little Ki.

10 Assessment rubric

	Emerging	Secure	Extending
Sensing	Prints an angle with help.	Shows accelerometer noise and gyro drift and explains why both are used.	Explains the 0.98 constant and why loop time (dt) matters.
Control and tuning	Pendulum stays up in the PID Lab with help.	Tunes the sim by method (P, then D, then I); recovers from a nudge.	Explains each gain's effect from response graphs and predicts changes.
Real robot	Robot stands briefly with given gains.	Robot balances; the tuning log shows one change at a time.	Robot recovers from a nudge and carries a load, or steers by shifting the setpoint.

11 Answers

Slide 2: Long broom or short?

> Long is easier: it falls more slowly, giving you time to react.

Slide 2: React too strongly? Too late?

> Too strongly: you overshoot and it falls the other way (too much P). Too late: it falls before you catch it (the loop is too slow).

Slide 2: Why can nobody balance a pencil?

> It is so short it falls faster than a human can react.

Slide 5: Why 0.98?

> It sets how long the filter trusts the gyro: at 100 Hz, about half a second before gravity's reading wins.

Exit ticket: shakes rapidly and falls.

> Lower Kp first: rapid shaking is over-correction. If it still wobbles, raise Kd, which brakes the motion.

12 Safety

- ! Lithium cells: charge only in the proper charger, never puncture, crush, short or heat them, and stop using any cell with a torn wrapper or swelling.
- ! Test over a soft surface, away from table edges.
- ! Keep fingers clear of the wheels; hold the robot by its body.
- ! Keep the 35° cutout in the code so a fallen robot stops its motors.
- ! Battery out before changing any wiring.

13 The complete angle code

Slide 12 shows the core of the filter. This version also reads the MPU6050 (the same I2C reads as EF-10), so it compiles as it is. Print the angle in loop() with a fixed 10 ms delay, then add the PID loop from slide 14.

```
#include <Wire.h>
const int MPU = 0x68;
float angle = 0;
unsigned long last = 0;

int read16() {                      // two bytes, high then low
  int hi = Wire.read();
  return (int16_t)((hi << 8) | Wire.read());
}

void setup() {
  Wire.begin();
  Wire.beginTransmission(MPU);
  Wire.write(0x6B); Wire.write(0);   // wake the MPU6050
  Wire.endTransmission();
  Serial.begin(9600);
  last = millis();
}

float readAngle() {
  Wire.beginTransmission(MPU);
  Wire.write(0x3B);                 // accelerometer X, high byte
  Wire.endTransmission(false);
  Wire.requestFrom(MPU, 14);
  int ax = read16(), ay = read16(), az = read16();
  read16();                         // temperature, not needed
  int gx = read16(); read16(); read16();
  float dt = (millis() - last) / 1000.0;
  last = millis();
  float aAng = atan2(ay, az) * 57.2958;   // tilt from gravity
  float gRate = gx / 131.0;               // degrees per second
  angle = 0.98 * (angle + gRate * dt) + 0.02 * aAng;
  return angle;
}

void loop() {
  Serial.println(readAngle());
  delay(10);                        // a steady 100 Hz
}
```